



Final Project: **FULBO** (Field **U**nified Localization and **B**all/player **O**bservation)

I308, Computer Vision (2025)

Name	Student ID
Ana Paula Tissera	35713
Valentino Arbelaiz Alberti	35496
Franco Amato de Lusarreta	35221

Professor: Gastón Castro

Teaching assistants: Esteban Uriza, Agustín Roca, Sebastián Bedin



Contents

1	Introduction	1
2	Dataset and characteristics	1
3	Methodology	2
3.1	Object detection	2
3.1.1	Player and referee detection	2
3.1.2	Ball detection	2
3.2	Tracking with ByteTrack	3
3.3	Player segmentation	3
3.3.1	Appearance extraction with DINOv2	3
3.3.2	Dimensionality reduction with UMAP	3
3.3.3	Team grouping with K-Means and per- <i>track</i> averaging	3
3.4	Keypoint detection with YOLOv8n-pose	3
3.5	Homography estimation	4
3.6	Projection	4
4	Results and analysis	4
4.1	Detection and tracking of players, referee and ball	4
4.2	Player grouping and team analysis	5
4.3	Keypoint detection	5
4.4	Homography estimation and projection	6
4.5	Discussion and interpretation	6
5	Conclusion and future work	6
6	References	7

Abstract

This work presents a system for detection, tracking and tactical analysis of football sequences using data from the SoccerNet dataset. The approach combines YOLO-based detection models for players, referees and the ball; a tracking module that associates detections over time using geometric and appearance information; similarity grouping through embedding *clustering* for team identification; and homography estimation from 29 and 57 keypoints in order to project the information onto the pitch plane. Quantitative evaluation tools were also developed, covering precision, recall, per-keypoint pixel-wise error and reprojection error, in order to validate the estimated homography. The results show solid performance on still images, whereas video sequences exhibit a degradation associated with changes in lighting, camera angles and *motion blur*, above all in the homography computation. The tracking stage behaved stably in sequences with reliable detections, keeping consistent identities over long intervals. Despite these limitations, the system produces interpretable geometric reconstructions and makes it possible to analyse the spatial occupation and dynamics of the match.

1 | Introduction

Football match analysis through computer vision has become an area of growing interest, both in academia and in real scouting, performance and tactical analytics applications. Reliably extracting structured information from video is, however, difficult on several fronts: the game is highly dynamic, occlusions are frequent, cameras vary visually from one broadcast to the next, and small objects such as the ball are hard to detect.

This work develops an end-to-end pipeline aimed at reconstructing the state of the game from video sequences. The system combines object detection, persistent tracking, visual embeddings, keypoint detection and geometric projection of the movement onto a plan view of the pitch.

Players, referees and the ball are detected with *YOLOv11*, splitting the frame into 4 patches for ball detection. *Frame-to-frame* tracking is handled by *ByteTrack*, which keeps identities stable even under partial occlusion or fluctuating confidence. To tell the two teams apart, high-dimensional visual descriptors are extracted with *DINOv2* and then reduced and grouped using *UMAP* and *K-Means*. Pitch keypoints are detected as well, in order to estimate a homography that maps image points onto the plane of play, which in turn allows the detected positions to be projected onto a tactical minimap.

The resulting pipeline is a solid base for more advanced *football analytics* tasks such as positional analysis, tactical pattern detection or automatic identification of relevant actions.

2 | Dataset and characteristics

Two datasets from the public *SoccerNet* platform were used to train and evaluate the pipeline.

The first one corresponds to the camera calibration task. It is made up of 20 028 images taken from 500 matches, annotated with the geometry of the pitch. Each image includes line segments representing the markings on the ground: touchlines and goal lines, penalty areas, the centre circle, the arcs and the goalposts, all defined in normalised coordinates $[0, 1]$. The data is split into training, validation and test sets.

The original SoccerNet labels are structured for semantic line detection tasks: they describe the geometry of the pitch through pairs of coordinates marking the start and the end of each segment. That format is incompatible with the YOLO pose estimation architecture, because the model requires a fixed list of keypoints in which every index of the output tensor invariably corresponds to one of them. The original data is also geometrically redundant, since the same vertex is represented several times as the endpoint of different lines. The labels therefore had

to be preprocessed and turned into an ordered set of coordinates.

Two different keypoint representations were produced. The first one, made up of 29 *keypoints*, follows the structure proposed by the *SoccerNet.Keypoints* module developed by Adit-jain [1]. This representation of the pitch has few points around midfield, so an occlusion or a missing point there causes problems. To address it, we looked for a way to generate and detect more points. The second representation, of 57 keypoints, was derived from the work of NikolasEnt [6], which adds extra points obtained from fine intersections, ellipse fitting and auxiliary structures of the field.

In both cases, the original segments were turned into points by computing intersections between lines, determining tangent points on curves, and geometrically reconstructing the centre circle and the penalty arcs. The coordinates were then normalised and labels were generated in the format required by YOLOv8 pose, including the attributes (x, y , visibility) for every point.

The second dataset belongs to the SoccerNet *tracking* task. It contains match clips captured exclusively by the main camera, split into 15 to 20 second clips at 1080p. The labels include *tracklets* for the players, the referees and the ball in every clip. For this work, the annotated bounding boxes were converted to YOLO format and the classes were

unified into player, referee and ball. The dataset was used both to train the detection model and to evaluate the ByteTrack-based tracking algorithm.

Preprocessing included normalising the annotation coordinates to the $[0, 1]$ range and adapting the SoccerNet JSON files to the structure required by the detection and tracking libraries.

3 | Methodology

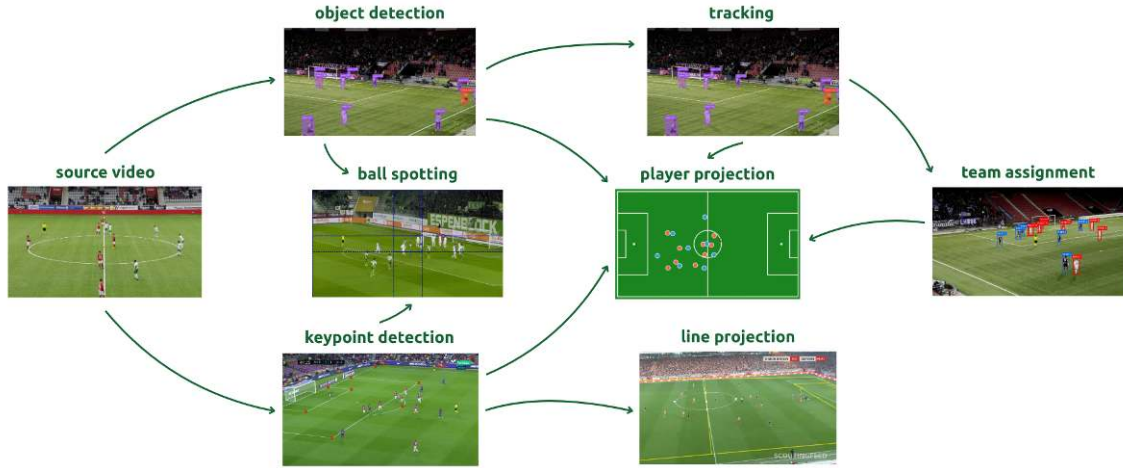


Figure 3.1: Task flow diagram.

3.1 | Object detection

Players, referees and the ball are detected with models from the *YOLO* family [3], trained on the *SoccerNet Tracking* dataset. For every frame of the video, the detector produces *bounding boxes* for the visible objects, together with a confidence score and the corresponding class. The predictions are processed with *non-maximum suppression* and then unified into a common format that feeds the tracking and team analysis stages.

3.1.1 | Player and referee detection

For players and referees, a *YOLOv11-m* base model was trained directly on the *SoccerNet Tracking* data. All three available classes (players, referee and ball) were included as *labels* in this training run. Because of the small size of the ball in the image and the large variability in its appearance (deformation and fast motion), however, the base model failed to detect it consistently.

In the final pipeline, therefore, the predictions of the base model are used only for the player and referee classes, in order to start the *tracking* stage and the later segmentation by team.

3.1.2 | Ball detection

A second, specialised model was trained from the weights of the base model (*fine-tuning*). A new dataset was built on top of the one used for the base model, keeping the ball as the only *label*.

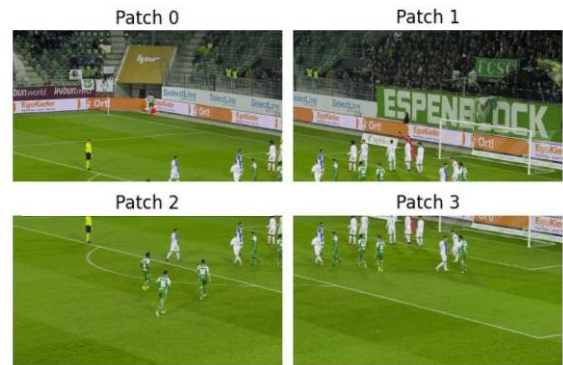


Figure 3.2: Splitting a frame into patches.

In order to improve the ball-size to image-size ratio, every frame was split into 2×2 patches with partial overlap between them (see Figure 3.2), producing new images in which the ball appears larger.

The same idea is used at inference time: the full frame is processed through a *slicing* scheme of overlapping patches, and the specialised ball model is applied to each of them. The detections produced

in each patch are rescaled back to the original coordinate system and merged with *non-maximum suppression*.

3.2 | Tracking with ByteTrack

Frame-to-frame tracking was addressed with *ByteTrack* [8], an object association method that uses both high-confidence and low-confidence detections to keep identities persistent in complex sequences. Starting from the boxes provided by the object detection model, the similarity between consecutive detections is computed using intersection over union (IoU). The resulting correspondences make it possible to reconstruct the trajectory of every player and referee over time.

3.3 | Player segmentation

The goal of this stage is to assign each player to one of the two teams based on their kit, using only the visual information in the crops produced by YOLO and the tracker.

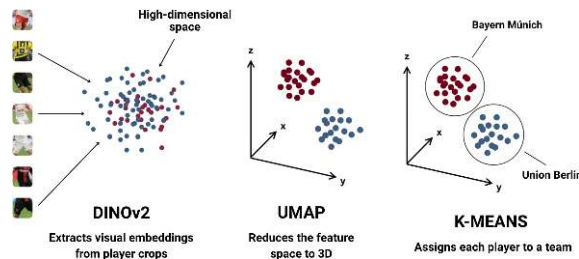


Figure 3.3: Overview of player segmentation: DINOv2 extracts appearance *embeddings*, UMAP reduces the dimensionality and K-Means groups them into two teams.

3.3.1 | Appearance extraction with DINOv2

To tell the players of each team apart, an *embedding* model, *DINOv2* [7], was used. A crop centred on the *bounding box* is extracted from every player detection and passed through the model, yielding a high-dimensional feature vector that captures colour and texture patterns of the clothing. These *embeddings* are collected over the whole clip, associated with the tracking identifier (*track id*) of each player.

3.3.2 | Dimensionality reduction with UMAP

The vectors produced by DINOv2 live in a high-dimensional space, so *UMAP* [5], a non-linear dimensionality reduction method, was applied. At this stage the representations in $\mathbb{R}^{768}$ are projected into a reduced space $\mathbb{R}^3$ through a transformation that preserves the local structure of the data, which makes grouping them easier.

3.3.3 | Team grouping with K-Means and per-track averaging

Teams are identified automatically with the *K-Means* algorithm [4]. Rather than clustering every crop individually, the average *embedding* across all of its crops over the video is computed first for each *track id*. This yields a single representative vector per player, to which UMAP and then K-Means with $K = 2$ are applied. The result is a map from *track id* to team, which is later used to colour every detection of that player consistently.

This scheme has two main advantages. Inference is more stable, since the team label does not change from frame to frame for the same *track*; and the computational cost of the grouping drops, because it works with one point per player instead of all the individual crops.

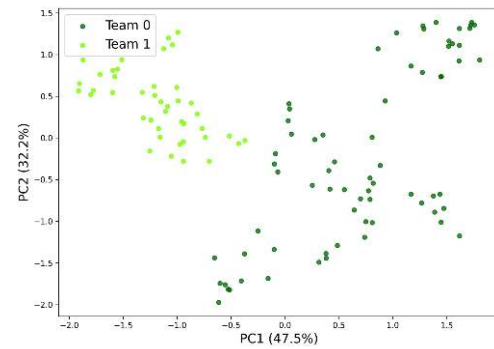


Figure 3.4: 2D visualisation of the *embeddings* projected with PCA. One point per *track*, corresponding to the average *embedding* of each player.

The teams contain more than 11 points, which is the expected maximum given that there are no more than 11 players per team. This happens because players "disappear" from the video and, when they come back into the scene, are assigned a new *tracker id*.

3.4 | Keypoint detection with YOLOv8n-pose

Reconstructing the geometry of the pitch requires a consistent set of keypoints. As explained earlier, these keypoints are not provided by SoccerNet directly: intersections, tangent points and geometric centres had to be computed to obtain the ground truth. Labels were then generated from those points in the *YOLOv8n pose* format, which made it possible to train the models.

Two main configurations were defined on top of the lightweight *YOLOv8n-pose* (Nano) architecture: one trained to detect the 29-keypoint dataset and another one for the 57-keypoint dataset.

A third variant was also trained using the *YOLOv8x-pose* (Extra Large) architecture. The aim of this run was to assess the impact of using a model with greater capacity and computational cost compared to the lightweight versions. Only the 29-keypoint dataset was used for this comparison, in order to analyse whether the increase in network depth produced a gain in accuracy.

3.5 | Homography estimation

With the keypoints detected in the image and their counterparts on the regulation model of the field (105 m × 68 m), a projective transformation was estimated using the OpenCV homography method [2]. The homography makes it possible to map any point

of the image onto the 2D plane of the playing field. To mitigate detection errors, the estimation is performed with RANSAC, which provides robustness against outliers and badly localised keypoints.

3.6 | Projection

Finally, the estimated homography is applied to the detections and trajectories of players, referees and the ball, moving them onto the plane of the pitch. Every frame is thereby translated into a set of absolute coordinates reflecting the position of each entity on the field. This two-dimensional map shows the projected positions in real time, which makes it easier to analyse formations, movements and relevant events of the match.

4 | Results and analysis

The experiments followed a hybrid methodology. The test set of each dataset was used to report standardised quantitative metrics, and real SoccerNet video sequences were also processed in order to qualitatively assess how the *pipeline* performs as a whole under dynamic conditions.

4.1 | Detection and tracking of players, referee and ball

This stage was evaluated by combining quantitative metrics on the SoccerNet test set with qualitative observations on video sequences. Broadly speaking, the system detects and follows players, referee and ball stably throughout the match, keeping consistent identities even under partial occlusion.

Table 4.1: Detection metrics for players and referees on the SoccerNet test set.

Class	Precision	Recall	F1
Player	0.891	0.908	0.899
Referee	0.942	0.586	0.722

Figure 4.1 shows the normalised confusion matrix for players and referees. On the original test set both classes are told apart correctly: players are always classified as *player*, and 96% of the referee instances are assigned to the right class. When the model is applied to leagues outside the SoccerNet dataset, however (the challenge only contained Bundesliga matches), generalisation turns out to be imperfect: when referees wear kits in colours that were not seen during training, that is, when they are not in yellow, the model tends to confuse them with players or the other way around, as can be seen in Figure 4.4a.

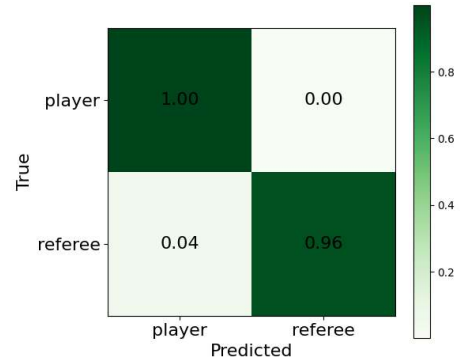


Figure 4.1: Normalised confusion matrix for the player and referee classes on the test set.

As for the ball, three variants of the detector were compared: the base model trained jointly with players and referee (*baseline*), the model specialised in the ball only (*ball*), and the specialised version with *slicing* inference over patches (*ball_sliced*). Figure 4.2 shows the *Precision–Confidence* curves. Over practically the whole range of thresholds, the specialised models reach a higher precision than the baseline, and the model with *slicing* achieves the best values.

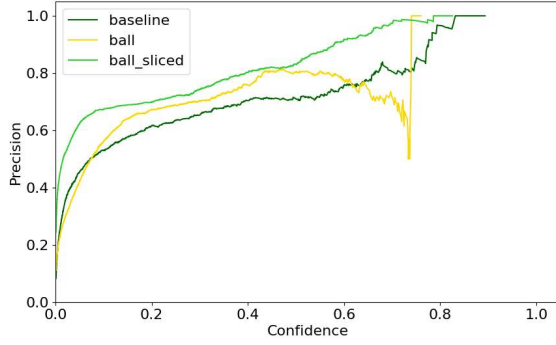


Figure 4.2: Precision-Confidence curves for the ball detection models: *baseline*, *ball* and *ball_sliced*.

4.2 | Player grouping and team analysis

Grouping by appearance was treated as an unsupervised learning problem, so the evaluation was carried out qualitatively. Starting from the visual embeddings obtained with DINOv2 and the per-track averaging scheme, the model groups the players into two clusters that consistently correspond to the two teams on the pitch.

In the sequences that were evaluated, the team assignment stayed stable over time, even under partial occlusion or changes in lighting. This makes it possible to colour every *track* with the colour of its team and to keep a clear separation between the two sides of the field.

Figure 4.3 shows an example frame with the final team segmentation, in which the players of each team are labelled correctly.



Figure 4.3: Team segmentation example: players coloured according to the assigned *cluster*.

4.3 | Keypoint detection

To evaluate keypoint detection accuracy, models trained with YOLOv8x to predict 29 and 57 points respectively were compared on the *test* set provided by SoccerNet. Two groups of metrics were reported in both cases:

- **Average per-point error (RMSE):** pixel-wise error for every keypoint marked as visible in the *ground truth*.
- **Visibility metrics:** *precision*, *recall*, true positives (TP), false positives (FP) and false

negatives (FN), treating any prediction with confidence > 0.4 as visible.

Both models show a consistent pattern in their individual errors. Keypoints placed in structurally well-defined areas, such as the corners of the penalty box and the upper corners of the field, have the lowest RMSE values, around 15–20 px in both models.

The points corresponding to midfield and to the lower touchline, on the other hand, were the most problematic in both architectures. The intersections of the halfway line with the upper and lower edges, as well as the points near the lower touchline, show the largest errors, reaching values close to 300–350 px. This is because these regions are not well-defined corners and are affected by *motion blur* during horizontal camera pans and by partial occlusion. As a consequence, both models are unstable in these areas, which later affects the homography estimation.

Comparison between the 29 and 57 keypoint models

Although the 57-keypoint model produces a more detailed geometric representation, its performance in visibility and consistency was slightly worse than that of the 29-keypoint model.

The visibility metrics for both models are summarised in Table 4.2.

Table 4.2: Comparison of visibility metrics between the 29 and 57 keypoint models.

Metric	29 KP	57 KP
Precision	0.805	0.779
Recall	0.856	0.803
TP	13821	34150
FP	3347	9697
FN	2317	8369

Note that the absolute TP, FP and FN values are not directly comparable across architectures, since the 57-keypoint model evaluates a larger number of points per image. For that reason, the comparison between models is based mainly on normalised metrics such as precision and recall.

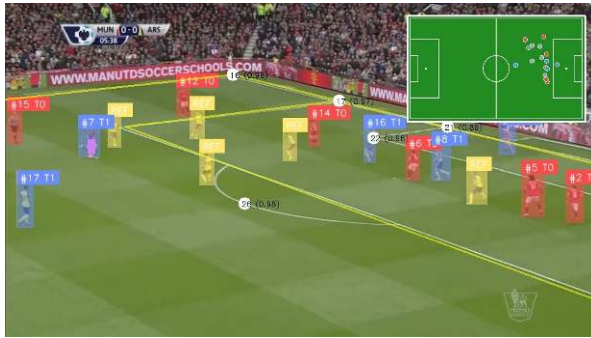
Overall, the 57-keypoint model produced a larger proportion of low-confidence points, which hurts frame-to-frame stability. The 29-keypoint model, although geometrically less expressive, gave more robust and stable predictions.

The final pipeline therefore uses the 29-keypoint model.

Despite the good results on the test set, a significant difference in performance was observed when the model was applied to continuous video sequences

coming from a different domain: matches from other leagues, different lighting conditions and camera angles that were not seen during training. Good results on still images notwithstanding, inference on video was highly unstable over time.

This can be seen in Figure 4.4. The poor performance is attributed to how different the images are compared to the training set, to the image quality degradation inherent to video, and to the *blur* introduced by camera pans.



(a) Result on the test set (still image).



(b) Result on video.

Figure 4.4: Performance comparison. (a) shows the accuracy under ideal conditions. (b) shows the instability and the loss of points caused by motion blur and low image quality.

4.4 | Homography estimation and projection

Homography estimation was directly tied to the performance of keypoint detection. The drop in keypoint detection accuracy on video had an immediate impact on the homography computation. Since the projection matrix is highly sensitive to the destination coordinates, it turned out to be very inconsistent over time. Visually this showed up as reprojection errors on the radar, where player positions jumped or fell out of alignment with the lines of the virtual pitch.

Even so, although the projection was noisy (especially during pans), the system recovered almost immediately once the camera held still or moved smoothly. Under those conditions the model was able to recover high-confidence detections, so the homography converged correctly and the jitter in the tactical view disappeared.

4.5 | Discussion and interpretation

The *pipeline* developed here demonstrates that detection, tracking and appearance analysis modules can be integrated to produce a metric representation of the game. Performance was asymmetric across the stages: object detection and team classification proved highly consistent and robust to occlusion, whereas the geometric reconstruction turned out to be critically sensitive to the quality of keypoint detection.

The results validate the effectiveness of the proposed approach, but they show that estimating the homography on video calls for additional considerations. Factors such as the symmetry of the pitch introduce positional ambiguities, and the frequent lack of visible points in tight shots makes it impossible to have the redundancy needed to filter out detection errors. This indicates that the stability of the metric system depends on several factors, not only on the spatial accuracy of the detector.

5 | Conclusion and future work

This work designed and implemented an end-to-end computer vision pipeline aimed at the tactical digitisation of football matches. The resulting system is a working tool that combines several architectures to turn a video signal into information about the match. Along the way, the effectiveness of modern detection and tracking models was validated: architectures such as YOLO and association algorithms such as ByteTrack are able to keep player identities even under partial occlusion and dense movement.

A significant contribution of this work was the team segmentation module based on visual *embeddings*. Using pre-trained vision models (DINOv2) together with dimensionality reduction (UMAP) and clustering (K-Means) proved to be a better strategy than traditional colour histogram methods. This approach separated the teams semantically with high accuracy and without any match-specific or league-specific retraining. The patch-based (slicing) inference strategy implemented for ball detection likewise gave impressive results when detecting and tracking small, fast-moving objects, even when they occupy a minimal fraction of the frame.

Evaluating the keypoint detection model on new video sequences, however, exposed critical generalisa-

tion challenges that had not shown up during controlled experimentation. The test videos did not belong to the same dataset used to train the keypoint models. This visual discrepancy between the training images and the new videos showed that the model was unable to generalise the geometric features of the pitch correctly to different leagues, stadiums or cameras. Together with the noise introduced by compression and camera motion, this caused fluctuations in the homography matrix that had not appeared during evaluation on the original test set.

As future work, we propose hardening the system by attacking the observed instability directly. A priority line of improvement is to analyse all the frames in order to assess the consistency of the homography across them and correct it in between, that is, to look at the behaviour over the whole video rather than at each frame and its neighbours only. It would also be interesting to add action detection models to the pipeline, so as to gain a better understanding of what is happening at each moment of the match.

6 | References

- [1] Adit-jain. Soccer_analysis: Soccernet_keypoints module, 2023. https://github.com/Adit-jain/Soccer_Analysis/tree/main/Data_utils/SoccerNet_Keypoints.
- [2] Gary Bradski and Adrian Kaehler. *Learning OpenCV*. O'Reilly, 2008.
- [3] Glenn Jocher et al. Ultralytics yolov8. 2023. <https://github.com/ultralytics/ultralytics>.
- [4] Stuart Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 1982.
- [5] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.
- [6] NikolasEnt. soccernet-calibration-sportlight, 2023. <https://github.com/NikolasEnt/soccernet-calibration-sportlight>.
- [7] Maxime Oquab, Tim Darcet, Theo Moutakanni, et al. Dinov2: Learning robust visual features without supervision. *arXiv:2304.07193*, 2023.
- [8] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, and Zehuan Yuan. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022.